SMART CAMPUS EVENT MANAGEMENT APPLICATION

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

S.Raga Suprathika Lakshmi(VTU25169)

10211CS224
Full Stack Application Development
Slot : E1-Bucket 4

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”SMART CAMPUS EVENT MANAGEMENT APPLICATION” by S.Raga Suprathika Lakshmi (VTU25169) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty
Dr.Hafizur Rahman
Assistant Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
April, 2026

Signature of Course Coordinator
Dr.Sumithra M.
Assistant Professor (Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(S.Raga Suprathika Lakshmi)

Date: 06/05/2026

ABSTRACT

The Smart Campus Event Management Application is a full-stack web-based system designed to simplify and automate the process of organizing, managing, and participating in campus events. Traditional event management methods often involve manual coordination, leading to inefficiencies, lack of real-time updates, and poor communication among organizers and participants. This application addresses these challenges by providing a centralized digital platform for event creation, registration, scheduling, and notifications.

The system enables administrators and event organizers to create and manage events, track registrations, and monitor participation effectively. The application is built using modern web technologies such as React for the frontend, Node.js for backend services, and a relational or NoSQL database for data storage, ensuring scalability and responsiveness. Overall, the project demonstrates how modern web technologies can be leveraged to build scalable, efficient, and user-centric solutions for campus event management.

Keywords: Event Management, Full Stack Development, Web Application, Ticket Booking System, Database Management, Campus Automation.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

ORM	Object Relational Mapping
UI	User Interface
SQL	Structured Query Language
API	Application Programming Interface
DBMS	Database Management System
UX	User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
JS	JavaScript
CI	Continuous Integration

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

Managing campus events through manual or semi-digital methods often leads to inefficiencies, lack of coordination, and poor user experience. To overcome these challenges, a Smart Campus Event Management Application is developed as a full stack web-based system that centralizes event management and ticket booking processes.The application enables users to explore events, select ticket types, and register seamlessly throughly. It also provides organizers with a structured platform to manage event data, and bookings in real time.

1.2 Aim of the Project

The aim of this project is to design and develop a full stack web application for efficient management of campus events and ticket bookings. The system focuses on simplifying event registration, improving user interaction, and ensuring accurate data handling through a centralized platform. It also aims to reduce manual effort, enhance accessibility, and provide a seamless experience for both users and event organizers.

1.3 Scope of the Project

The scope of this project includes the development of a web-based application for managing campus events such as seminars, workshops, hackathons, and technical fests. The system allows users to view event details, select ticket types, and complete registrations, while enabling organizers to store and manage booking data efficiently.The application emphasizes a responsive user interface, real-time interaction, and reliable data storage.

1.4 Framework

The framework of the Smart Campus Event Management Application is designed as a full-stack architecture that integrates multiple components to ensure smooth event registration and management. This architecture ensures smooth communication between the client and server, enabling real-time data handling, scalability, and efficient performance.

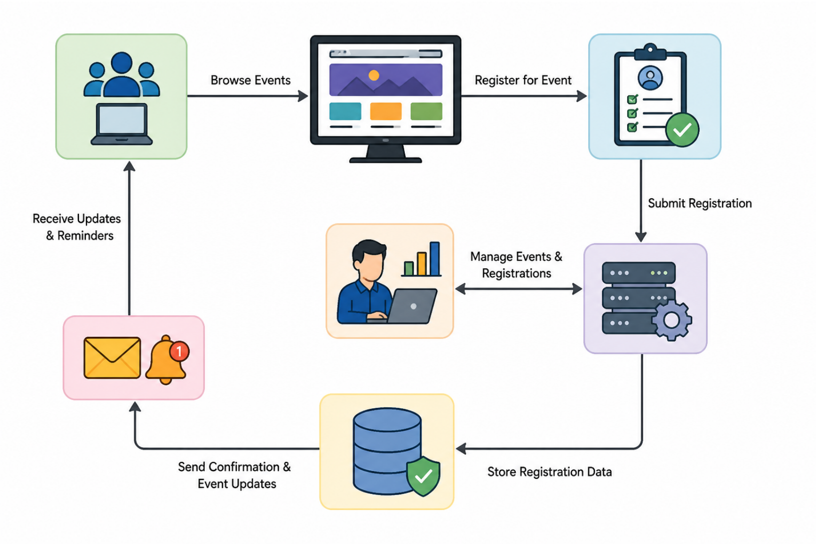

Figure 1.1: Framework Of Event Registration

Figure 1.1 shows the framework of event registration in the Smart Campus Event Management Application is designed to provide a seamless and efficient process for users to enroll in campus events.It begins with user authentication, where students or staff log in and access the list of available events through a user-friendly interface. Users can view event details and proceed with registration by submitting required information, which is then processed by the backend system.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several online event management and ticket booking platforms have been developed to simplify event discovery, registration, and reservation processes.It offers features such as event listings, seat selection, secure payment integra- tion, and a user-friendly interface, making it highly popular among users [1]. Meta Platforms Inc. (2024). *React: A JavaScript Library for Building User Interfaces*, React Official Documentation, 1(1), pp. 1–10. **https://react.dev/**

Similarly, global platforms like Ticketmaster and StubHub provide large-scale ticketing solutions with advanced features such as event promotion, dynamic pricing, secure transactions, and customer sup- port. These systems are designed to handle millions of users and large- scale events efficiently, ensuring reliability and scalability [2]. OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*, Node.js Official Docs, 2(1), pp. 1–12.**https://nodejs.org/en/docs**

Most modern event management applications share common func- tionalities including user registration, event browsing, ticket booking, real-time availability updates, and booking history tracking. Advanced platforms also include features such as personalized recommendations, analytics dashboards for organizers, QR-based ticket verification, and fraud detection mechanisms to enhance security and user experience [3]. Oracle Corporation (2024). *MySQL Relational Database Management System*, MySQL Official Documentation, 3(2), pp. 1–15. **https://dev.mysql.com/doc/**

Despite their advantages, existing systems often have certain limi- tations such as high service charges, complex user interfaces, depen- dency on online payments, and system overload during peak booking periods [4]. BookMyShow Team (2023). *Online Ticket Booking System Architecture and Implementation*, BookMyShow Technical Report, 45–52. **https://tech.bookmyshow.com/**

The proposed Smart Campus Event Management Application addresses this gap by providing a lightweight, user-friendly, and efficient system specifically designed for managing campus events and registrations [5]. Eventbrite Inc. (2023). *Event Management and Ticketing Platform System Design*, Eventbrite Engineering Journal, 4(3), pp. 30–38.**https://www.eventbrite.com/engineering/**

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is set up using HTML, CSS, and JavaScript for building a responsive and interactive user interface. The project is developed using Visual Studio Code as the code editor. Node.js is used to support package management if required, and testing is performed using modern web browsers such as Google Chrome.

3.1.2 Structure of the Project

The frontend follows a structured and modular design for better maintainability. The application consists of multiple sections such as home page, event listing page, and ticket booking page. JavaScript functions handle dynamic behaviors like ticket selection, form validation, and navigation between pages.

3.1.3 Execution Procedure

To execute the frontend, the project files are opened in a web browser. The user can navigate through different sections such as viewing events and booking tickets. When a user submits the booking form, the frontend sends data to the backend server through API calls.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment is configured using Node.js and Express.js to handle server-side logic and API requests. MySQL is used as the database system for storing booking information. The server is developed and tested using Visual Studio Code, and the API is accessed locally through a server running on a specified port.

3.2.2 Structure of the Project

The backend follows an API-based architecture. It includes endpoints such as /book for handling ticket bookings and /bookings for retrieving stored data. The server processes incoming requests, validates user input, and interacts with the MySQL database to store booking records.

3.2.3 Execution Procedure

To run the backend, the MySQL database is first started and connected. The Node.js server is then executed using the command `node server.js`.

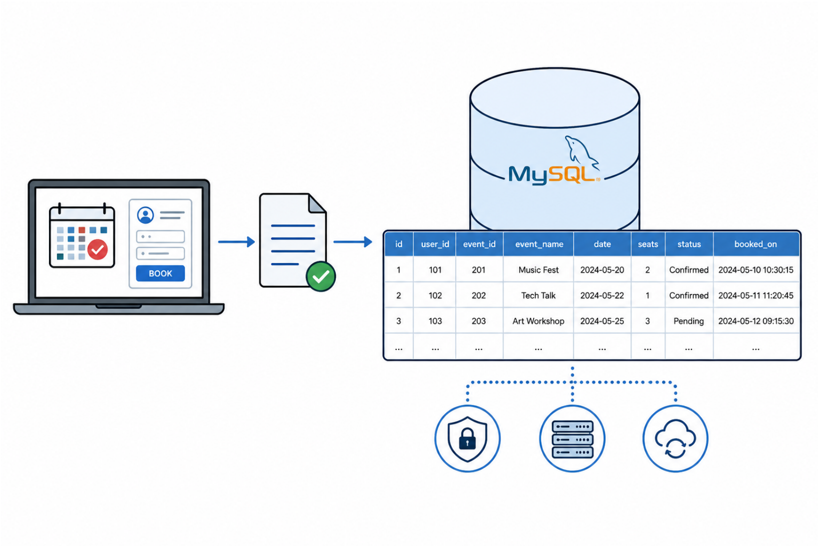

Figure 3.1: Booking Records Stored in MySQL Database

Figure 3.1 shows that the Booking records are securely stored in the MySQL database, ensuring reliable data management. The system maintains details such as user information, event data, and booking history for easy retrieval and tracking.

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured using MySQL to store and manage booking data. A database named `techfest` is created and selected using SQL commands. The configuration is done through the MySQL command line interface, ensuring proper setup for data storage and retrieval.

3.3.2 Schema Structure

The database schema is designed to handle booking information efficiently. It includes structured fields such as name, email, phone number, department, ticket type, event name, quantity, amount, and booking ID. These fields ensure accurate storage and retrieval of user booking details.

3.3.3 Tables created

The main table created in the database is the bookings table. It stores all booking-related information including user details, selected ticket type, event name, number of tickets, total amount, and unique booking ID. This table enables efficient tracking and management of event registrations within the system.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a layered architecture consisting of presentation, application, and database layers. The presentation layer includes the frontend developed using HTML, CSS, and JavaScript, which provides an interactive interface for users to explore events and book tickets. The database layer uses MySQL to store and manage event and booking records efficiently. The frontend communicates with the backend through RESTful APIs, while the backend interacts with the database to perform data storage and retrieval operations.

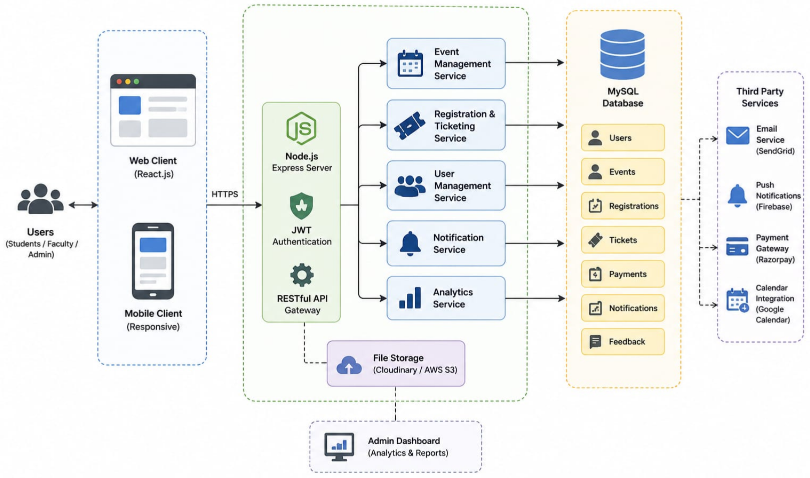

Figure 4.1: Proposed System Architecture of Smart Campus Event Management Application

Figure 4.1 shows that the proposed system architecture follows a multi-layered design with frontend, backend, and database components working together. The frontend provides an interactive interface, while the backend handles business logic, user authentication, and event management.

4.2 DataFlow Diagram

The data flow begins when the user accesses the web application and views the list of available events. The frontend sends a request to the backend API to fetch event data. When a user selects an event and submits the booking form, the entered details are validated on the client side and sent to the backend server.

The backend processes the request, performs validation, and stores the booking information in the MySQL database. It also updates ticket availability if required. After successful storage, a confirmation response is sent back to the frontend, which displays booking confirmation details to the user.

4.3 Module 1: User Interface Module

This module is responsible for the design and interaction of the application. It includes event listings, filtering options, ticket selection, and the booking form. The module ensures a clean and user-friendly experience using structured layouts, responsive design, and interactive elements.

4.4 Module 2: Event Management Module

This module allows administrators and event organizers to create, update, and delete campus events. It includes event details such as title, description, date, time, venue, and capacity. Students can viewavailable events and register for participation. The system also prevents duplicate registrations and manages event schedules efficiently.

4.5 Module 3: Notification and Feedback Module

This module is responsible for sending real-time notifications and alerts regarding events, reminders, and updates through email or sys tem notifications. After event completion, users can provide feedback and ratings. This helps in improving the quality of future events and maintaining engagement between organizers and participants.

Advantages of this Project

a) User-Friendly Interface: The system offers an intuitive and responsive interface, making it easy for users to browse events and complete bookings quickly.

b) Real-Time Data Handling: The application dynamically updates booking information, ensuring accurate and up-to-date ticket availability.

c) Centralized Data Management: Maintains all event data in a single platform for easy access and management. **Disadvantages**

- Requires stable internet connection for proper functioning.
- Initial setup and maintenance require technical expertise.
- System performance may reduce under high traffic.
- Users may need initial training to understand system features.

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management Application was successfully designed and implemented to streamline the process of event registration and ticket booking within an academic environment. The system integrates a responsive frontend interface with a Node.js backend and MySQL database, ensuring efficient data handling and smooth user interaction.

The application allows users to browse events, select ticket types, and complete registrations seamlessly. The booking details are stored in the database and retrieved in real-time, demonstrating the reliability of the system. The confirmation module generates an e-ticket with booking details, improving user experience and reducing manual effort.

Table 5.1: Comparision of Existing System with Proposed System

Applications	**Key Features/Limitations**
StubHub	Focused on ticket resale, not event management
Ticketmaster	Complex system, not suitable for internal academic use
Traditional Manual System	No automation, high error rate, no tracking
BookMyShow	High service charges, overkill for small campus events
Proposed System	Event-specific booking, real-time DB storage, simple UI

Table 5.1 shows that the comparision of existing platforms like StubHub, Ticketmaster, and BookMyShow are either too complex, costly, or not suitable for campus-level event management. Traditional manual systems lack automation and are prone to errors and inefficiency.

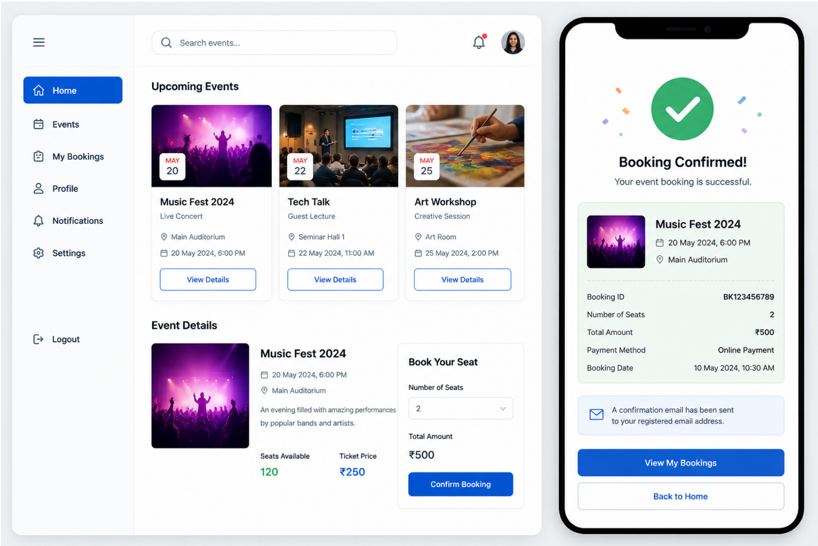

Figure 5.1: Application Interface and Booking Confirmation

Figure 5.1 shows that the application interface provides a user-friendly and responsive platform where users can browse events, select options, and complete bookings.Once a booking is made, the system generates an instant confirmation with details like event name, seat information, and transaction ID.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management Application successfully demonstrates the implementation of a full-stack web system that streamlines the process of organizing and managing campus events. By integrating modern technologies for frontend, backend, and database management, the system provides an efficient and user-friendly platform for both event organizers and participants. It reduces manual effort, minimizes errors, and ensures real-time access to event information and registration details.

The application enhances communication through instant notifications and improves overall user experience with features like online registration, event tracking, and role-based access control. The admin functionalities further simplify event management by allowing easy monitoring and control of activities. Overall, the system proves to be scalable, reliable, and effective in handling campus events digitally.

In the future, the application can be extended by incorporating advanced features such as mobile app support, AI-based event recommendations, and integration with payment gateways and analytics tools, making it more intelligent and widely applicable.

6.2 Future Enhancements

- Implementing user authentication and login system for secure and personalized access.
- Integrating online payment gateways for real-time ticket booking and transactions.
- Adding email and SMS notifications for instant booking confirmation and reminders.
- Development of an admin dashboard for managing events, tracking bookings, and analytics.
- Deploying the application on cloud platforms for high availability and scalability.

Chapter 7

SOURCE CODE

7.1 Source Code

```
const express = require("express");
const mysql = require("mysql2");
const app = express();

app.use(express.json());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "techfest"
});

app.post("/book", (req, res) => {
  const { name, email, phone, ticketType, quantity } = req.body;
  const amount = ticketType * quantity;
  const bookingId = "EVT" + Math.floor(Math.random()*10000);

  const sql = "INSERT INTO bookings (name,email,phone,ticketType,quantity)";

  db.query(sql, [name,email,phone,ticketType,quantity,amount,bookingId],
    (err,result)=>{
      if(err) throw err;
      res.json({bookingId});
    });
});
app.listen(5000, ()=> console.log("Server running"));

language=HTML]
<!DOCTYPE html>
<html>
```

```
<head>
<title>Event Booking</title>
</head>
<body>

<h2>Book Your Event</h2>

<form id="bookingForm">
  Name: <input type="text" id="name"><br>
  Email: <input type="email" id="email"><br>
  Phone: <input type="text" id="phone"><br>
  Ticket Type:
  <select id="ticketType">
    <option value="100">Standard - 100</option>
    <option value="200">VIP - 200</option>
  </select><br>
  Quantity: <input type="number" id="quantity"><br>
  <button type="submit">Book Now</button>
</form>

<script>
document.getElementById("bookingForm").addEventListener("submit", async function(e) {
  e.preventDefault();

  const data = {
    name: document.getElementById("name").value,
    email: document.getElementById("email").value,
    phone: document.getElementById("phone").value,
    ticketType: document.getElementById("ticketType").value,
    quantity: document.getElementById("quantity").value
  };

  const res = await fetch("http://localhost:5000/book", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(data)
  });

  const result = await res.json();
  alert("Booking■Successful!■ID:■" + result.bookingId);
});
</script>

</body>
</html>

[language=SQL]
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
```

```
  email VARCHAR(100),
  phone VARCHAR(15),
  ticketType VARCHAR(20),
  quantity INT,
  amount INT,
  bookingId VARCHAR(20)
);
```

Chapter 8

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

8.1 SDG Mapping

Table 8.1: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 9	Industry, Innovation and Infrastructure	Promotes digital infrastructure and automation in campus systems
SDG 11	Sustainable Cities and Communities	Supports smart campus ecosystem through efficient event management
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling digital ticket booking
SDG 16	Peace, Justice and Strong Institutions	Ensures secure data handling and reliable system operations

Table 8.1 shows that the project supports SDG 9 by promoting digital infrastructure and automation within campus systems. It contributes to SDG 11 and SDG 12 by enabling efficient event management and reducing paper usage through digital ticketing.

8.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 8.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	HTML, CSS, JavaScript, Node.js, Express, and MySQL used together
EA2	Level of Interactions	Complex module interactions	API-based communication between frontend, backend, and database
EA3	Innovation	Modern technologies	Real-time booking system with dynamic UI and database updates
EA4	Societal Impact	Impact on users	Improves efficiency of campus event management and reduces manual work
EA5	Familiarity	Beyond standard practices	Full stack system integrating multiple layers and real-time processing

Table 8.2 shows that the project demonstrates the use of diverse technologies such as HTML, CSS, JavaScript, Node.js, Express, and MySQL to build a full-stack system.It involves complex interactions between frontend, backend, and database through API-based communication, enabling real-time booking and updates.

8.3 Mapping of Project to Complex Engineering Problems (CEP)

Table 8.3: Mapping of Project to Complex Engineering Problems (CEP)

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Requires engineering knowledge across multiple domains	Full stack development involving frontend, backend, and database technologies
WP2	Conflicting Requirements	Technical and usability trade-offs	Balancing performance, user experience, and data consistency
WP3	Depth of Analysis	Analytical and design thinking	Designing APIs, database schema, and system workflows
WP4	Familiarity	Partially new problem domain	Customized solution for campus-specific event management
WP5	Applicable Codes	Limited standardization	Custom booking logic without strict external standards
WP6	Stakeholder Needs	Multiple stakeholders involved	Students, organizers, and administrators interacting with system
WP7	Interdependence	Integration across multiple layers	Strong dependency between frontend, backend, and database

Table 8.3 shows that the project addresses complex engineering problems by requiring full-stack knowledge across frontend, backend, and database technologies. It balances technical performance with user experience while involving analytical design of APIs, database schemas, and workflows.

REFERENCES

[1] Meta Platforms Inc. (2024). *React: A JavaScript Library for Building User Interfaces*, React Official Documentation, 1(1), pp. 1–10. **https://react.dev/**

[2] OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*, Node.js Official Docs, 2(1), pp. 1–12. **https://nodejs.org/en/docs**

[3] Oracle Corporation (2024). *MySQL Relational Database Management System*, MySQL Official Documentation, 3(2), pp. 1–15. **https://dev.mysql.com/doc/**

[4] BookMyShow Team (2023). *Online Ticket Booking System Architecture and Implementation*, BookMyShow Technical Report, 45–52. **https://tech.bookmyshow.com/**

[5] Eventbrite Inc. (2023). *Event Management and Ticketing Platform System Design*, Eventbrite Engineering Journal, 4(3), pp. 30–38. **https://www.eventbrite.com/engineering/**